TEK-UP
Private Higher School
of Technologies & Engineering

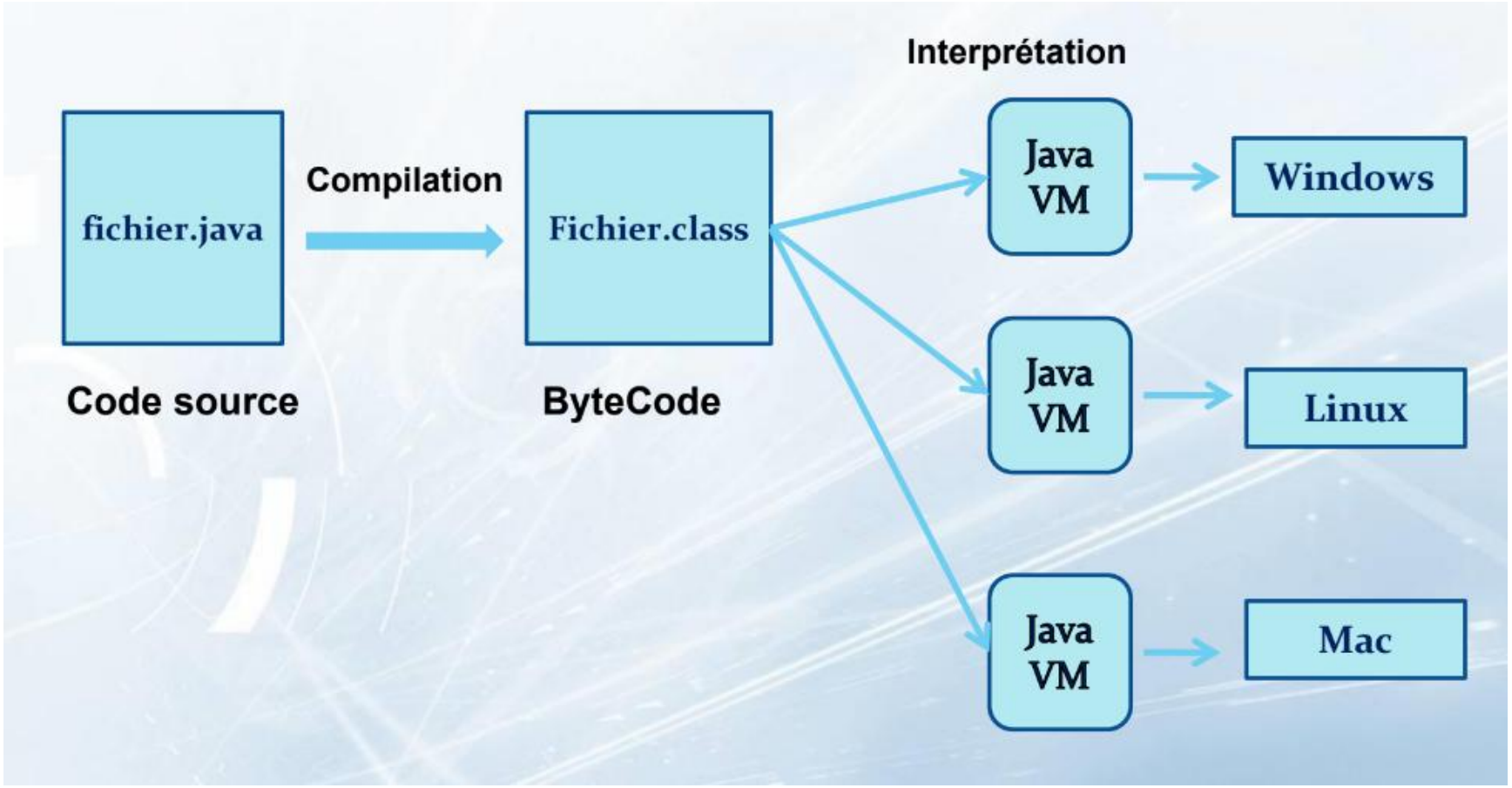
Eléments de base de Java

Qu’est-ce que Java ?

❑ **Java:**

- est un **langage de programmation** et une **plate-forme**.
  - ❖ **Plate-forme**: Tout environnement **matériel ou logiciel** dans lequel un programme s’exécute est connu sous le nom de **plate-forme**.
- Java est un langage de **haut niveau**, **robuste** et **orienté objet**.
- a été développé par **Sun Microsystems** (qui est maintenant la filiale d’Oracle) en **1995**. **James Gosling** est connu comme le père de Java.
- est conçu pour être **portable**, c'est-à-dire qu'il peut être exécuté sur différentes plateformes, telles que **Windows**, **Mac OS** et **Linux**, sans avoir besoin d'être recompilé.

Qu’est-ce que Java ?



Qu'est-ce que Java ?

❑ **Java:**

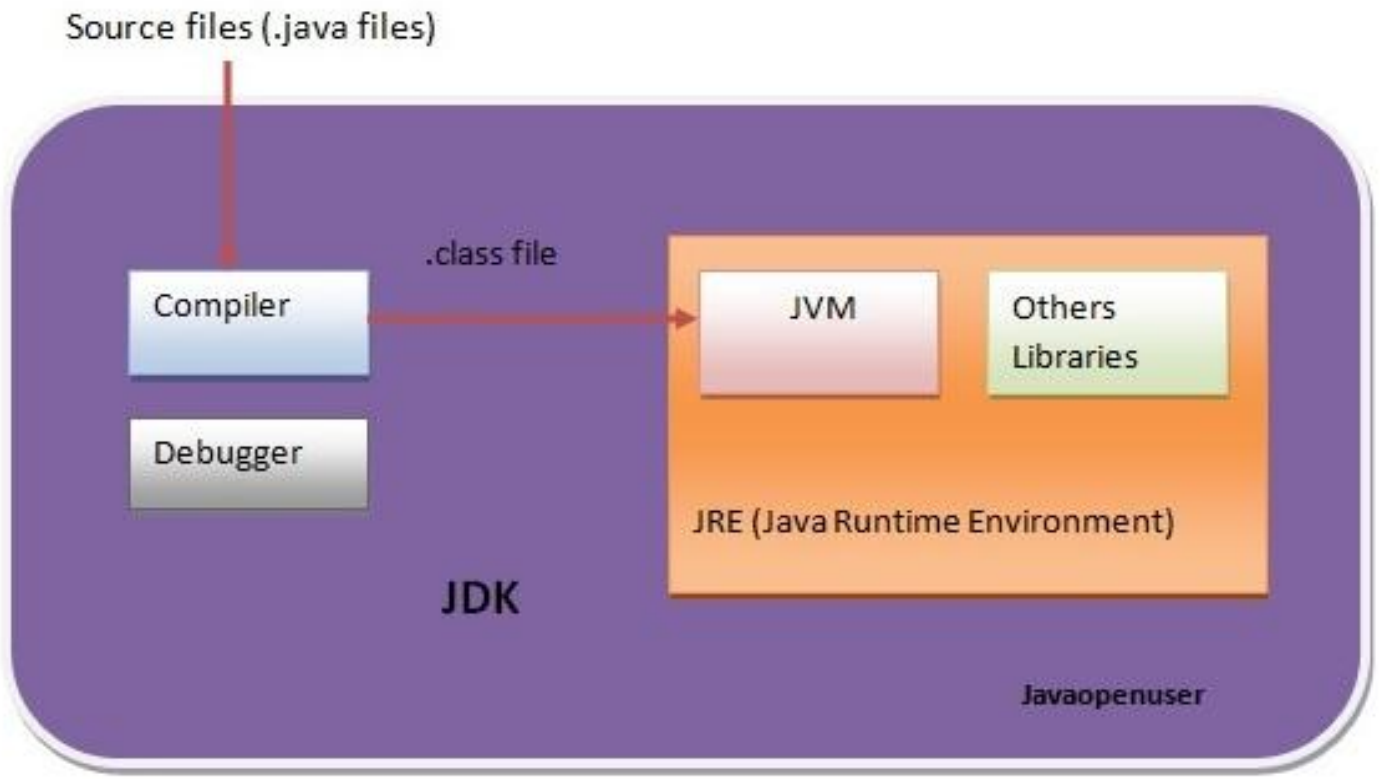
- est également connu pour **sa sécurité**, **sa performance** et **sa fiabilité**, ce qui le rend populaire pour le développement de logiciels d'entreprise, d'applications mobiles, de jeux, de systèmes d'exploitation et de nombreuses autres applications.

- Le code source **n'est pas traduit** directement dans le langage de l'ordinateur. Il est d'abord traduit dans un langage appelé "**bytecode**", langage d'une machine virtuelle (JVM – Java Virtual Machine) définie par Sun.

  - ❖ le **bytecode Java** est un format binaire intermédiaire qui est exécuté par la machine virtuelle Java

  ✓ Le **bytecode** Java est constitué d'instructions qui sont exécutées par la JVM.

Qu’est-ce que Java ?



Différence entre JDK, JRE et JVM

❑ JVM:

- Est une machine abstraite.
- On l'appelle une machine virtuelle car elle n'existe pas physiquement.
- Il s'agit d'une spécification qui fournit un environnement d'exécution dans lequel le **bytecode** Java peut être exécuté.
- Il peut également exécuter les programmes écrits dans d'autres langages et compilés en bytecode Java.
- La JVM effectue **les tâches principales** suivantes :
  - ❖ Charge le code
  - ❖ Vérifie le code
  - ❖ Exécute le code
  - ❖ Fournit un environnement d'exécution

Différence entre JDK, JRE et JVM

❑ JVM:

- JVM fournit des définitions pour :
  - ❖ Zone mémoire
  - ❖ Format de fichier de classe
  - ❖ Ensemble de registres
  - ❖ Tas d'ordures ménagères
  - ❖ Rapport d'erreur fatale, etc.

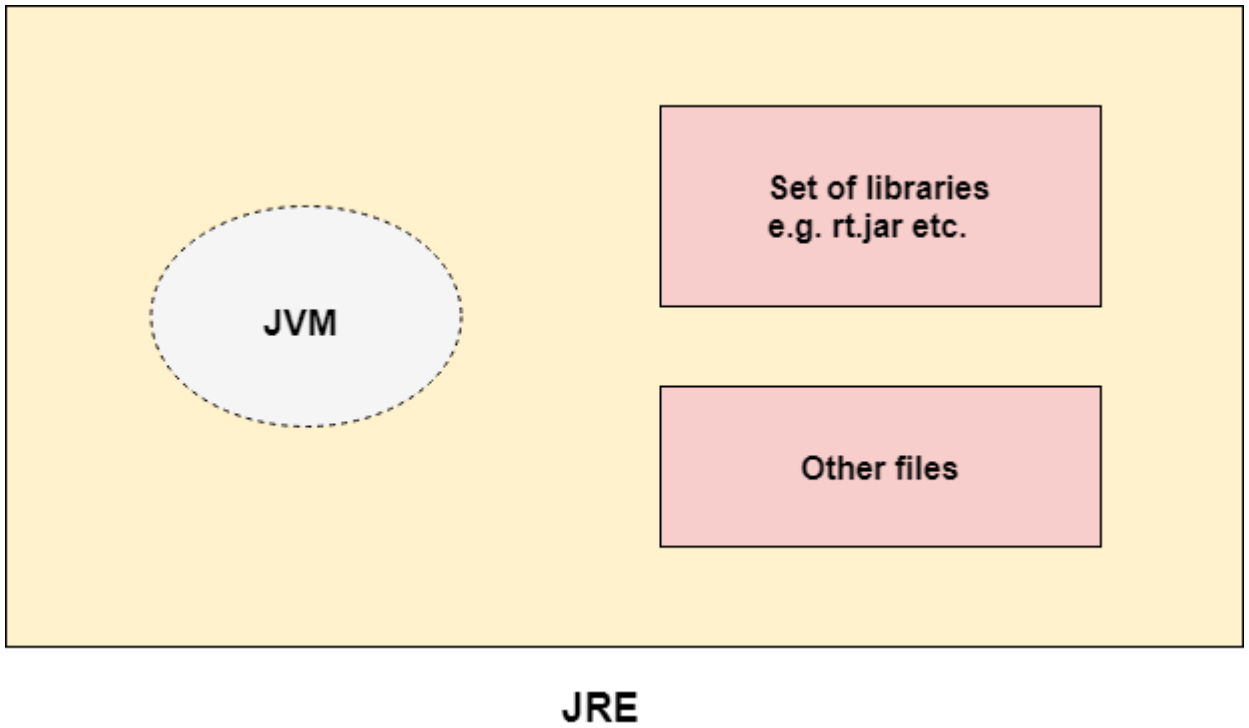
❑ Compilateur Just-In-Time (JIT) :

- Il est utilisé pour améliorer les performances.
- JIT compile des parties du byte code qui ont des fonctionnalités similaires en même temps, et réduit donc le temps nécessaire à la compilation.
- Ici, le terme « compilateur » fait référence à un traducteur du jeu d'instructions d'une machine virtuelle Java (JVM) vers le jeu d'instructions d'un processeur spécifique.

Différence entre JDK, JRE et JVM

❑ JRE:

- ➢ Est l'acronyme de Java Runtime Environment. Il est également écrit comme Java RTE.
- ➢ Est un ensemble **d'outils logiciels** utilisés pour **développer** des applications Java.
- ➢ Est utilisé pour **fournir l'environnement d'exécution**.
- ➢ Il existe physiquement.
- ➢ Contient **un ensemble de bibliothèques** + **autres fichiers que JVM** utilise au moment de l'exécution.



Différence entre JDK, JRE et JVM

❑ JDK:

- Est l'acronyme de **Java Development Kit**.
- Est un environnement de développement logiciel utilisé pour développer des applications et des applets Java.
- Il existe physiquement.
- Contient des outils de développement JRE +.
- Est une implémentation de l'une des plates-formes Java:

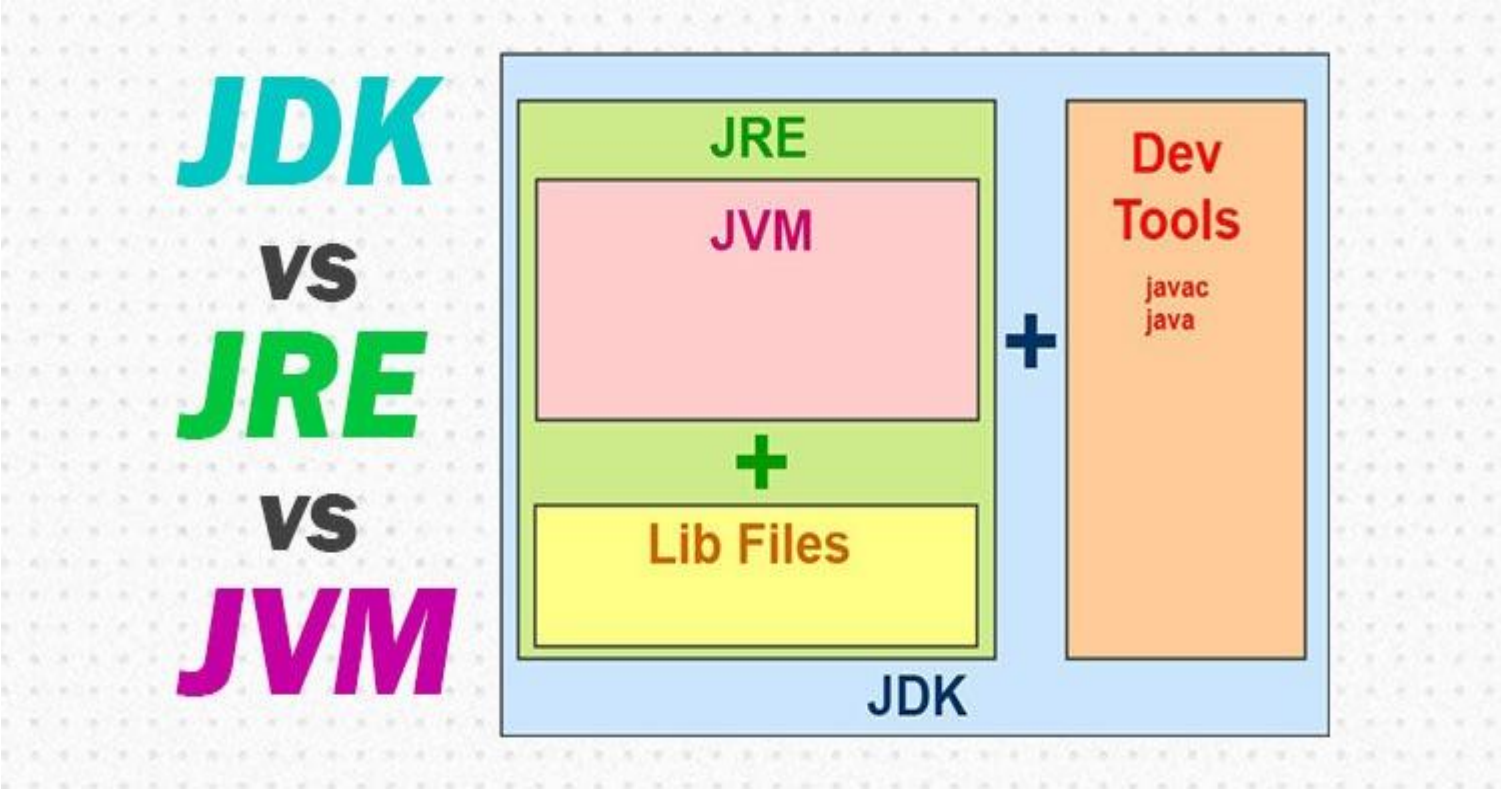
  - ❖ Plate-forme Java Standard Edition
  - ❖ Plate-forme Java Enterprise Edition
  - ❖ Micro Edition Java Platform

Différence entre JDK, JRE et JVM

❑ JDK:

- Contient :
  - ❖ une machine virtuelle Java (JVM) privée
  - ❖ quelques autres ressources telles qu'un interpréteur/chargeur (java), un compilateur (javac), un archiveur (jar), un générateur de documentation (Javadoc), etc.
  - ➔ Pour terminer le développement d'une application Java.

Différence entre JDK, JRE et JVM



En résumé, le JDK est nécessaire pour développer des applications Java, le JRE est nécessaire pour exécuter des applications Java et la JVM est responsable de l'exécution du code Java.

Plates-formes / Éditions Java

❑ Il existe **quatre plates-formes** ou éditions de Java :

1. Java SE (Java Standard Edition)

➢ C'est une plate-forme de programmation Java. Il comprend des API de programmation Java telles que **java.lang**, **java.io**, **java.net**, **java.util**, **java.sql**, **java.math**, etc.

- ❖ **javax.swing :** package récent proposant des composants « légers » pour la création d'interfaces graphiques
- ❖ **java.net :** sockets, URL
- ❖ **java.applet :** les applets sur le web
- ❖ **java.util :** structures de données classiques
- ❖ **java.io :** entrées / sorties (clavier, fichiers,…)
- ❖ **java.lang :** gestion des chaînes de caractères (**String**), interaction avec l'OS (**System**), programmation parallèle (**Thread**).

Plates-formes / Éditions Java

- **java.sql :** fournit le package JDBC
- **java.awt :** interfaces graphiques, images et dessins

- Il comprend des sujets principaux tels que la chaîne, Regex, l'exception, les classes internes, le multithreading, le flux d'E/S, la mise en réseau, AWT, le swing, la réflexion, la collection, etc.

2. Java EE (Java Enterprise Edition)

- Il s'agit d'une plate-forme d'entreprise qui est principalement utilisée pour développer des applications Web et d'entreprise.

- Il est construit sur la plate-forme Java SE. Il comprend des sujets tels que Servlet, JSP, Web Services, EJB, JPA, etc.

L'API de Java

3. Java ME (Java Micro Edition)

- Il s'agit d'une micro-plateforme dédiée aux applications mobiles.

4. JavaFX

- Il est utilisé pour développer des applications Internet riches. Il utilise une interface utilisateur légère.

Syntaxe du langage Java

❑ Premier programme Java

class HelloWorld ➔Cette ligne utilise mot-clé "class" pour déclarer qu'une nouvelle classe est définie.

```
{
   /* Votre programme commence par un appel à main().
   Affiche "Hello, World" dans la fenêtre du terminal. */

   public static void main(String args[])
   {
      System.out.println("Hello, World");
   }
}
```

Syntaxe du langage Java

❑ Premier programme Java

- ❖ **public**: Pour que la machine virtuelle Java puisse exécuter la méthode depuis n'importe où.

- ❖ **static**: la méthode principale doit être appelée sans objet.
  Les modificateurs public et static peuvent être écrits dans l'un ou l'autre ordre.

- ❖ **void**: la méthode principale ne renvoie rien.

- ❖ **main()**: Nom configuré dans la machine virtuelle Java.

- ❖ **String[]:** la méthode principale accepte un seul argument: un tableau d'éléments de type String.

Syntaxe du langage Java

❑ Premier programme Java

❖ System.out.println("Hello, World");

- ➢ Cette ligne affiche la chaîne «Hello, World» suivie d'une nouvelle ligne à l'écran.
- ➢ La sortie est en réalité réalisée par la méthode println() intégrée.
- ➢ "System" est une classe prédéfinie qui fournit un accès au système,
- ➢ et "out" est la variable de type flux de sortie connecté à la console.

Syntaxe du langage Java

❑ Les commentaires

/* commentaire sur une ou plusieurs lignes */

➔ Identiques à ceux existant dans le langage C

// commentaire de fin de ligne

➔Identiques à ceux existant en C/C++

❑ Instructions, blocs

- ❖ Un bloc d'instructions commence par { et se termine par }
- ❖ Les instructions Java se terminent par un ;
- ❖ Les espaces, tabulations, sauts de ligne sont autorisés. Cela permet de présenter un code plus lisible.

Syntaxe du langage Java

❑ Types de données en Java

- Il existe deux types de données en Java:

1. Types de données primitifs : Les types de données primitifs incluent boolean, char, byte, short, int, long, float et double.

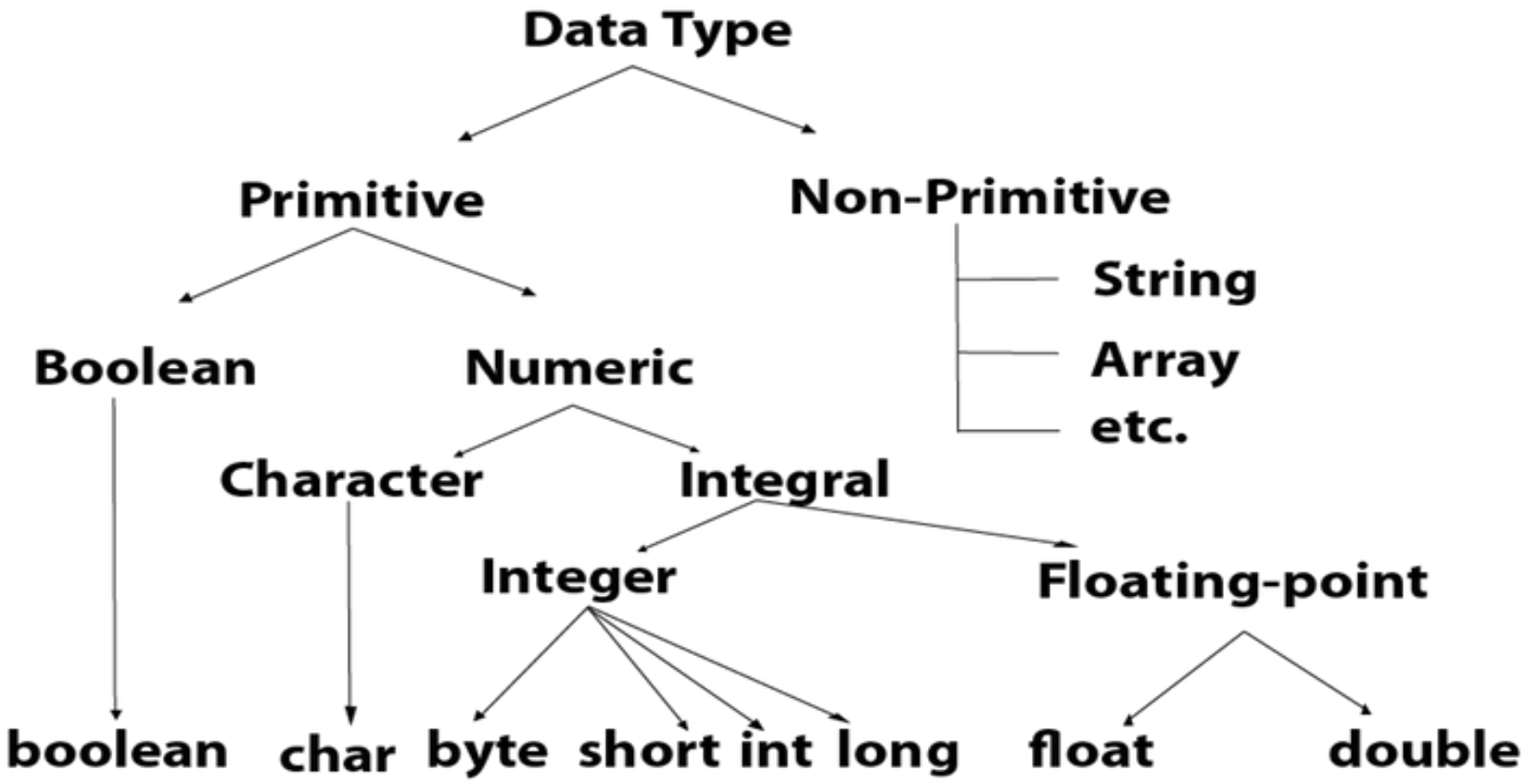
2. Types de données non primitifs : Les types de données non primitifs incluent les classes, les interfaces et les tableaux.

✓ Types de données primitifs Java:

- Ce sont les types de données les plus basiques disponibles en langage Java.

Syntaxe du langage Java

❑ Types de données en Java



Syntaxe du langage Java

❑ Opération arithmétiques:

Opération	Utilisation
+	Op1+op2
-	Op1-op2
*	Op1*op2
/	Op1/op2
%	Op1%op2

Syntaxe du langage Java

❑ Les opérateurs standards:

❖ Les opérateurs de comparaison:

opération	Description
==	EGALITE
!=	INEGALITE
<	INFERIEUR
<=	INFERIEUR OU EGAL
>	SUPERIEUR
>=	SUPERIEUR OU EGAL

❖ Les opérateurs logiques:

- ✓ && ET
- ✓ || OU
- ✓ ! NON

Syntaxe du langage Java

❑ Les structures de contrôles:

- :

Alternative: if-else
Choix Multiple: switch-case
Itérations: for, while, do-while

Les structures de contrôle

❑ les structures conditionnelles ou alternatives:

❖ If - else:

Syntaxe

```
If(condition)
{
Instruction 1;
…….
}
else
{
Instruction 2;
…….
}
```

```
public class Max
 {
public static void main(String args[])
{
if (nb1 > nb2)
System.out.println("l'entier le plus grand est "+ nb1);
else
System.out.println("l'entier le plus grand est "+ nb2);
}
}
```

Les structures de contrôle

❑ les structures conditionnelles ou alternatives:

❖ if - else if - ... – else:

➢ En combinant plusieurs structures **if - else** en une expression nous obtenons une structure qui est très courante pour prendre des décisions entre plusieurs alternatives.:

```
if ( <cond1> )
<instruction 1>
else if (<cond2>)
< instruction 2>
else if (<cond3>)
< instruction 3>
else if (<condN>)
< instruction N>
else
< instruction N+1>
```

Les structures de contrôle

❑ Structure à choix multiple: Switch

*** syntaxe:**

```
Switch(expression)
{
Case valeur1: instruction1;
break;
Case valeur2: instruction2;
break;
……..
Case valeurN: instructionN;
break;
Default:  Autre_instruction;
}
```

- ❖ Chaque <**instruction**> suivie éventuellement de **break**; pour arrêter l'exécution des autres cas.
- ❖ Les <**valeur**> peuvent être constante ou expression constante entière ou caractère.
- ❖ L'expression est évaluée et est comparée à toutes les constantes.
- ❖ Dès que sa valeur est égale à une de ces constantes, l'exécution reprend au choix correspondant.
- ❖ **Default** est optionnel et est exécuté quand aucun des case ne correspond à l'expression qui apparaît dans **le switch (expression)**. C'est généralement une bonne idée d'y mettre le signalement d'une erreur.

Les structures de contrôle

❑ les structures répétitives ou itératives:

➢ Il existe trois sortes de structures répétitives**: while**, **do while** et **for.**

- **While**

Syntaxe:

while (condition)
{
bloc d'instructions ;
}
=> tant que condition est vraie (≠0) on exécute bloc d'instructions. Si la condition fournit la valeur zéro, l'exécution continue avec l'instruction qui suit le bloc d'instructions.

Les structures de contrôle

❑ les structures répétitives ou itératives:

- **DO-While**

La structure **do - while** est semblable à la structure **while**, avec la différence suivante :
* **while** évalue la condition avant d'exécuter le bloc d'instructions,
* **do - while** évalue la condition après avoir exécuté le bloc d'instructions. Ainsi le bloc d'instructions est exécuté au moins une fois.

Syntaxe:

```
 do
 {
 bloc d'instructions ;
} while(condition) ;
```

Les structures de contrôle

❑ les structures répétitives ou itératives:

- **for**

➢ La structure **for** en C est plus générale et beaucoup plus puissante.

syntaxe:

for(expr1;expr2;expr3)
{ bloc d'instructions ; }

expr1 est évaluée une fois avant le passage dans la boucle, elle est utilisée pour initialiser les données de la boucle.
expr2 est évaluée à chaque passage de la boucle, elle est utilisée pour savoir si la boucle est répétée ou non (c'est une condition de répétition, et non d'arrêt).
expr3 est évaluée à la fin de chaque passage de la boucle, elle est utilisée pour réinitialiser les données de la boucle.

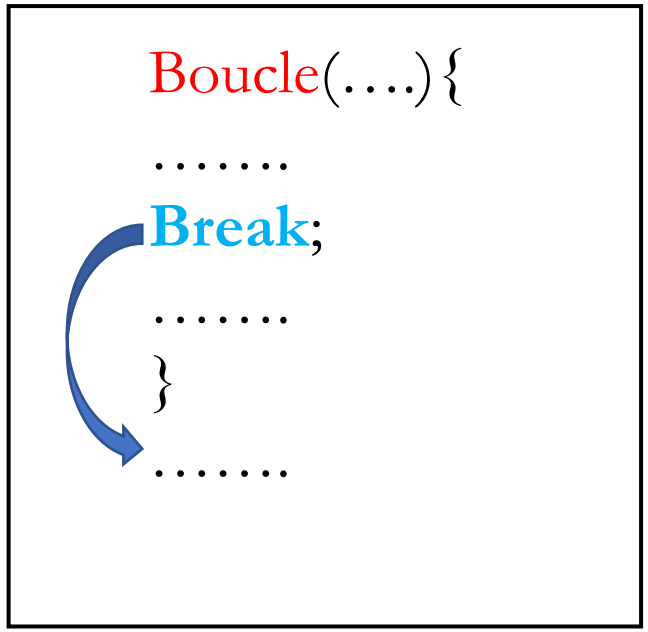
Les structures de contrôle

❑ les structures répétitives ou itératives:

• L'instruction break

- L'instruction **break** permet d' arrêter le déroulement d'une boucle, et de passer à l'instruction qui suit cette boucle.

Syntaxe:



Les structures de contrôle

❑ les structures répétitives ou itératives:

• L'instruction continue

- L'instruction **continue** permet d' ignorer l'itération actuelle de la boucle et de passer à l'itération suivante.

Syntaxe:

```
Boucle(….){
…….
continue;
……..// l'itération
actuelle
}
…….
```

Les entrées sorties

❑ Affichage sur console:

System.out.print(chaîne de caractères à afficher)
System.out.println(chaîne de caractères à afficher)

❖ chaîne de caractères peut être :

✓ **Une constante chaîne de caractères (String)**

```
System.out.println("coucou");
```

✓ **Une expression de type String**

```
System.out.println(age);
```

✓ **une combinaison (concaténation) de constantes et d'expressions de type String. La concaténation est exprimée à l'aide de l'opérateur +**

```
System.out.println("L'age de la personne est " + age + " son
poids " + poids);
```

Les entrées sorties

❑ Lecture à partir du clavier :

❖ La classe Scanner simplifie la lecture de données sur l'entrée standard clavier.

✓ Pour utiliser la classe Scanner, il faut d'abord l'importer:

```
import java.util.Scanner;
```

✓ Ensuite il faut créer un objet de la classe Scanner :

```
Scanner sc = new Scanner(System.in);
```

Les entrées sorties

❑ Lecture à partir du clavier :

Méthodes de l'objet String	Désignation
String next()	Donnée de la classe String qui forme un mot,
String nextLine()	Donnée de la classe String qui forme une ligne,
boolean nextBoolean()	Donnée booléenne,
int nextInt()	Donnée entière de type int,
double nextDouble()	Donnée réelle de type double.

Les entrées sorties

❑ Lecture à partir du clavier :

Exemple

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner( System.in );
        int n1 ,n2, somme;
        System.out.print(" Donner le premier entier: " );
        n1 = sc.nextInt();
        System.out.print(" Donner un deuxième entier: " );
        n2 = sc.nextInt();
        somme = n1 + n2;
        System.out.println( "Somme est "+ somme );
       // sc.close();
}}
```

Les tableaux

- Un tableau est une structure de données qui stocke une série de n valeurs de même type (élémentaire ou objet)

  - L'accès à chaque valeur se fait avec un indice entier
  - Soit « tab » un tableau d'entiers
  - **tab[i]** est le ième entier du tableau
  - En Java i varie de 0 à n-1 (n est la taille du tableau)
  - L'accès à la case « n » provoquera lève une exception du type **java.lang.arrayIndexOutOfBoundsException**.

Les tableaux

❑ Déclaration et Initialisation

❖ Pour définir un tableau d'entier on peut écrire:

int[] a; ou **int a[];**

➔ Cette instruction ne déclare que la variable « a »

❖ L'opérateur « new » crée le tableau:

a=new int[100];

int a[]=new int[100];

❖ On peut aussi créer un tableau et l'initialiser simultanément avec ses valeurs

int[] nbrPremiers= {2, 3, 5, 7};

Cette syntaxe est employée pour réinitialiser un tableau sans créer une nouvelle variable

nbrPremiers =new int[] {11, 13, 17, 19};

Les tableaux

❑ Taille et Parcours d'un tableau

❖ On peut récupérer la taille d'un tableau à l'aide de **nomduTableau.length**

❖ Pour remplir ou parcourir un tableau nous pouvons utiliser une boucle:

```
int[] nbrPremiers= {2, 3, 5, 7};
for (int i = 0; i < nbrPremiers.length; i++)
System.out.println(nbrPremiers[i]);
```

❑ Parcours d'un tableau avec for

❖ Structure de contrôle introduite par JDK5, adaptée aux collections, aux tableaux et aux chaînes.

```
int t [ ] = {1,2,3,4};
for (int v : t) System.out.println (v) ;
```

❖ la variable v prendra successivement les différentes valeurs du tableau t. On obtiendra le même résultat qu'en utilisant :

```
for (int i = 0; i < t.length; i++)
System.out.println(t[i]);
```

Les tableaux

❑ Copie des tableaux

❖ Pour copier toutes les valeur d'un tableau dans un autre, il faut employer la méthode **arraycopy** de la classe **System**

System.arraycopy(source, indiceSource, cible, indiceCible, compte)

❖ **Exemple:**

```
public class Main {
    public static void main(String[] args) {
        int[] ta= {2, 3, 5, 7};
        int[] tb={1,10,100,1000,1010,1100};
        System.arraycopy(ta,1,tb,2, 3);
        for (int i = 0; i < tb.length; i++)
            System.out.print(tb[i]+" ");
    }}
```

```
C:\Users\USER\.jdks\openjdk-19.0.2\bin\java.exe
1 10 3 5 7 1100
Process finished with exit code 0
```

Matrice

❑ Déclaration:

❖ Déclarer et d'utiliser des tableaux à plusieurs dimensions:

float tableau[][] = new float[10][10];

❖ Exemple:

```
public class Main {
   public static void main(String[] args) {
// Déclaration d'une matrice à deux dimensions
      int[][] maMatrice = { {1, 2, 3}, {4, 5, 6}, {7, 8, 9} };
// Affichage de la matrice
      for (int i = 0; i < maMatrice.length; i++) {
         for (int j = 0; j < maMatrice[i].length; j++) {
            System.out.print(maMatrice[i][j] + " ");
         }
         System.out.println();
      } }}
```

Matrice

❖ Exemple 2:

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
         System.out.print("Entrez le nombre de lignes de la matrice : ");
       int n = sc.nextInt();
       System.out.print("Entrez le nombre de colonnes de la matrice : ");
       int m = sc.nextInt();
       int[][] maMatrice = new int[n][m];
       for (int i = 0; i < n; i++) {
           for (int j = 0; j < m; j++) {
               System.out.print("Entrez la valeur pour l'élément [" + i + "][" + j + "] : ");
               maMatrice[i][j] = sc.nextInt();
           }
       }
```

Matrice

❖ **Suite de l'exemple 2:**

```
// Affichage de la matrice
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                System.out.print(maMatrice[i][j] + " ");
            }
            System.out.println();
        }

    }
}
```

Matrice

Application:

Écrire un programme qui :

- Déclare un tableau T de 10 valeurs entières,
- Calcule et affiche le minimum, le maximum et la moyenne de T.

Solution:

Les chaines de caractères

❖ Le type de données String (chaîne de caractère) est une classe.

- Une chaîne de type String est un objet.
- Les chaînes de caractère figurent entre guillemets.
- Un objet String n'est utilisable qu'à travers les méthodes de la classe String.

Il est impossible de modifier le contenu d'un objet String construit à partir d'une constante. Cependant, il est possible d'utiliser les méthodes qui renvoient une chaîne pour modifier le contenu de la chaîne.

String s1 ="Bonjour";
String s2 =« tout le monde";
Concaténer des chaînes :
String s3 = s1.concat(s2); **ou** String s3 = s1+ " " + s2;

Les chaines de caractères

Comparer des chaînes :

if (str1 == str2) ... Ou
if (str1.equals(str2)) ... Ou str1.compareTo(str2); ➔ Cette méthode retourne 0 si les deux chaînes sont égales, une valeur négative si **str1** est plus petit que **str2**, ou une valeur positive si **str2** est plus petit que **str1**.

Remplacer :

```
String texte = "Java Java Java";
texte = texte.replace('a','o');
```

Détermination de la longueur d'une chaîne:

La méthode **length()** permet de déterminer la longueur d'une chaîne.

Exemple:
```
String texte = " texte ";
int longueur = texte.length();
```

Les chaines de caractères

Modification de la casse d'une chaîne:

toUpperCase() permet d'obtenir une chaîne tout en majuscule

toLowerCase() permet d'obtenir une chaîne tout en minuscule.

Exemple: String texte = " texte ";

String textemaj = texte.toUpperCase();

Application:

Écrire un programme Java qui demande à l'utilisateur de saisir son nom et son âge, puis affiche un message de salutation personnalisé en fonction de l'âge de l'utilisateur.

- Si l'utilisateur a moins de 18 ans, le programme doit afficher "Bonjour [nom de l'utilisateur], vous êtes mineur.".
- Si l'utilisateur a entre 18 et 64 ans (inclus), le programme doit afficher "Bonjour [nom de l'utilisateur], vous êtes majeur.".
- Si l'utilisateur a 65 ans ou plus, le programme doit afficher "Bonjour [nom de l'utilisateur], vous êtes senior."